NTNU
Kunnskap for en bedre verden

Department of Mechanical and Industrial Engineering

TMM4935 - Industrial ICT, Master's Thesis

Development of video streaming tool for Remote Train Operations

Author:
Einar Thingstad Myhrvold

Student number:
543951

Supervisor:
Nils O.E. Olsson

Date: 18.03.2026

Preface

This master thesis

The development of ATO at NTNU has been in progress for multiple years and is now in the remote operation phase. Master students at the Department of Mechanical and Industrial Engineering has steadily worked on this project. In the fall of 2024 and spring of 2025 it was from Emilia Kozarevic, and I will continue her and NTNU's research on the topic.

Subject & knowledge listed below has been used.

- TBA4225 - Railway Engineering 2
- IT2810 - Web Development
- TDT4240 - Software Architecture
- TDT4120 - Algorithms and Data Structures

I would like to end the preface by thanking Nils Olsson for the work of supervising me during this semester and the effort he has put in to help me write this paper. I also would like to thank Patrick Urassa (NTNU) and Tomas Rosberg from VTI for discussing and the insight in the project.

Abstract

Norwegian

Table of Contents

List of Figures iv

List of Tables iv

1 Introduction 1

1.1 Background and Motivation . . . 1

1.2 Problem . . . 1

1.3 Objectives . . . 2

1.4 Scope and Limitations . . . 2

1.5 Outline of Structure . . . 2

1.6 System Overview . . . 3

1.6.1 Components . . . 3

2 Theory 4

2.1 ATO . . . 4

2.1.1 Grades of Automation . . . 4

2.1.2 Role of Remote Driving in Automated Railway Systems . . . 4

2.1.3 Remote Driving in GoA Levels . . . 4

2.2 Video Streaming Fundamentals . . . 4

2.2.1 Codecs . . . 4

2.2.2 RTP . . . 4

2.2.3 UDP . . . 4

2.3 GStreamer . . . 4

2.4 Latency Sources in Streaming Pipeline . . . 5

2.4.1 Capturing . . . 5

2.4.2 Encoding . . . 5

2.4.3 Transmitting . . . 5

2.4.4 Decoding . . . 5

2.4.5 Rendering . . . 5

2.5 Cellular Communications . . . 5

2.5.1 5G URLLC . . . 5

2.5.2 Wi-Fi . . . 5

3 System Design 6

3.1 Requirements . . . 6

3.2 Architecture 6
3.3 Pipeline Design 8
3.4 Timestamps and Latency Measurement Design 8
3.5 Wi-Fi Network Setup 8

4 Methodology **9**
4.1 Literature and Knowledge acquisition 9
4.1.1 Search Strategy and Databases 9
4.1.2 Inclusion Criteria 9
4.1.3 Search Queries and Refinement 10
4.2 Validity 10
4.3 Reliability 10
4.4 Research Design 10
4.5 Development 11
4.5.1 GStreamer Pipeline Development 11
4.6 Experimental Methodology 11
4.6.1 Latency Measurement 11

5 Results **12**

6 Conclusion **13**
6.1 Evaluation of latency 13

List of Figures

1 Physical Setup of Project 6
2 Application and Transport layer Setup of Project 7
3 Sequence Diagram between Sender and Camera before active Stream 8

List of Tables

1 Setup for IP and Port Connection 7
2 Inclusion Criteria for Literature Review 10

1 Introduction

The chapter of Introduction will contain text about purpose, research questions and a small bulk about history and the level of digitalisation we have reached right now.

1.1 Background and Motivation

As the railway sector transitions toward higher Grades of Automation (GoA), particularly GoA 3 and GoA 4, remote driving becomes an increasingly critical operational capability. Remote driving is not only a natural consequence of reduced onboard personnel at higher GoA levels, but serves as a mandatory safety layer within the ATO architecture enabling operators to intervene when automated systems encounter situations they cannot resolve independently, such as unexpected track conditions or degraded operational modes [source].

A fundamental requirement for safe remote driving is the availability of reliable, low-latency video streaming. When the operator is physically separated from the train, the video feed becomes the primary source of situational awareness, and its quality and responsiveness directly determine whether timely and safe intervention is possible [source]. Industry experience further demonstrates that remote operation can support a range of practical tasks including shunting, depot movements, and vehicle preparation provided that the communication link meets sufficient performance criteria [source].

Understanding the relationship between latency and human perception is equally important in this context. Research on human factors in remote operation indicates that video delay, bitrate, and stream stability all influence an operator's ability to detect obstacles, interpret dynamic scenes, and execute corrective actions under time pressure [source]. Identifying the latency boundaries within which safe remote operation remains feasible is therefore a key step toward enabling higher automation levels in railway systems.

This thesis is developed within an ongoing research collaboration led by NTNU with the help of VTI and other European railway organizations, contributing to the technical groundwork required to make remote train driving a safe and practical reality in future railway deployments.

1.2 Problem

The current remote train operation setup relies on a commercial off-the-shelf product, Voysys, which imposes significant limitations on the project's ability to test and develop the system further. As a closed external solution, it restricts access to the underlying pipeline, making it difficult to adjust configurations, measure internal latency, or experiment with different streaming approaches. In addition, licensing such a product over the course of a multi-year research project represents a considerable and ongoing cost [source].

At the same time, there is limited existing research that addresses video streaming specifically in the context of remote train operation. Most work on low-latency video transmission is either too general or focused on other domains, leaving a gap when it comes to understanding what configurations and trade-offs are most suitable for railway use cases [source].

Building a custom streaming solution also comes with its own set of challenges. Network conditions such as signal strength, interference, and varying load introduce unpredictable variability in transmission quality. Balancing video quality against latency is a non-trivial trade-off, and testing these trade-offs on real hardware in a realistic setup adds further complexity. Without a flexible and open pipeline, it is difficult to isolate and measure where latency is introduced and how different configurations affect overall system performance.

1.3 Objectives

The main objective of this thesis is to design and implement a low-latency video streaming pipeline tailored specifically for remote train operation, serving as a practical replacement for the current commercial solution. The pipeline should be functional, flexible, and open enough to support continued development and testing within the project. More specifically, the objectives are to:

1. O1: Develop a working GStreamer-based streaming pipeline that meets the low-latency requirements of remote train operation
2. O2: Implement a latency measurement framework that allows end-to-end delay to be captured, logged, and compared across different configurations
3. O3: Evaluate the pipeline across multiple configurations, including different codecs, payloaders, and network settings, to identify trade-offs between latency and video quality
4. O4: Design the system to be dynamic and extensible, allowing additional cameras and new configurations to be added with minimal effort
5. O5: Deliver a stable and usable prototype that is good enough for practical use during the current development period and serves as a foundation for future work

The result should be a system that gives the project full control over the streaming pipeline, removes dependency on external licensed software, and produces reliable data to guide future configuration decisions.

1.4 Scope and Limitations

Write more about this later in the process.

1.5 Outline of Structure

The remainder of this thesis is organized as follows:

Chapter 2 - Theory presents the theoretical background underlying the work. This includes an overview of Automatic Train Operation and the role of remote driving in higher GoA levels, fundamentals of video streaming such as codecs, RTP, and UDP, an introduction to GStreamer, the main sources of latency in a streaming pipeline, and relevant aspects of cellular and Wi-Fi communication.

Chapter 3 - System Design describes the design decisions made in developing the streaming system. This covers the requirements, overall architecture, pipeline design, the approach to timestamp-based latency measurement, and the Wi-Fi network setup used in testing.

Chapter 4 - Methodology explains how the research was conducted, including the research design, development process, GStreamer pipeline development, and the experimental and latency measurement methodology.

Chapter 5 - Experiment and Results presents the experimental findings, including a comparison of different GStreamer configurations, latency measurements per configuration, the effect of Wi-Fi conditions, and the trade-offs between video quality and latency.

Chapter 6 - Discussion interprets and reflects on the results in relation to the objectives and broader context of remote train operation.

Chapter 7 - Conclusion summarizes the key findings and contributions of the thesis and outlines directions for future work.

1.6 System Overview

A section to go through the system as it stands.

1.6.1 Components

A overview of the components in use. Also mentioning alternatives to the ones we have in use.

2 Theory

This chapter presents the theoretical background needed to understand the work carried out in this thesis. It covers the principles of Automatic Train Operation and the role of remote driving within that framework, the fundamentals of video streaming including codecs and transport protocols, an introduction to GStreamer as the core technology used in the pipeline, the main sources of latency in a streaming system, and relevant characteristics of the communication technologies used in testing.

2.1 ATO

2.1.1 Grades of Automation

2.1.2 Role of Remote Driving in Automated Railway Systems

2.1.3 Remote Driving in GoA Levels

2.2 Video Streaming Fundamentals

2.2.1 Codecs

2.2.2 RTP

2.2.3 UDP

2.3 GStreamer

GStreamer is an open-source multimedia framework that allows developers to build pipelines for processing and transmitting audio and video streams. It is widely used in both research and industry due to its flexibility, broad codec support, and active community [source]. Unlike closed commercial solutions, GStreamer provides full access to every stage of the streaming process, making it well suited for experimentation and performance tuning.

The core concept in GStreamer is the pipeline, which is a chain of processing elements connected in sequence. Each element performs a specific task, such as capturing video from a camera, encoding the stream, packeting it for network transmission, or rendering it on a display. Data flows through the pipeline from one element to the next, and each element can be configured independently. This modular design makes it straightforward to swap out components, such as replacing one codec with another, without rebuilding the entire pipeline [source].

Pipelines in GStreamer can be constructed and launched directly from the command line using the gst-launch-1.0 tool, or built programmatically using languages such as Python through the GStreamer Python bindings. For this project, both approaches are used, with Python providing the flexibility to control pipeline behaviour, handle events, and integrate latency measurement logic alongside the streaming process [source].

What makes GStreamer particularly relevant for this thesis is the level of control and transparency it offers. Every element in the pipeline can be monitored and tuned, latency can be measured at specific points, and configurations can be changed and tested systematically. This makes it a practical and powerful foundation for building a low-latency video streaming system for remote train operation.

2.4 Latency Sources in Streaming Pipeline

2.4.1 Capturing

2.4.2 Encoding

2.4.3 Transmitting

2.4.4 Decoding

2.4.5 Rendering

2.5 Cellular Communications

2.5.1 5G URLLC

2.5.2 Wi-Fi

3 System Design

This chapter describes the design of the video streaming system developed in this thesis. It outlines the requirements that shaped the system, the overall architecture, the pipeline design, the approach used for timestamp-based latency measurement, and the Wi-Fi network setup used during testing. Together these sections document the reasoning behind the key design decisions made throughout the development process.

3.1 Requirements

The requirements for the system are shaped by the operational context of remote train operation, where reliability and low latency are critical. Unlike a controlled lab setting, a moving train introduces real-world challenges such as varying network conditions, physical vibrations, and the need for the system to perform consistently during operation.

The key requirements identified for the system are:

Low latency: The pipeline must deliver video with as little end-to-end delay as possible, as latency directly affects the remote operator's ability to perceive and react to the environment safely.

Reliability: The system must maintain a stable stream under varying network conditions, as interruptions or degraded quality during operation could have serious safety implications.

Multi-camera support: The system must be capable of handling multiple camera streams simultaneously, reflecting the practical needs of a remote driving setup where several viewing angles are required.

Deployable on a moving train: The system must function in a real train environment, accounting for the network and physical conditions that come with that setting.

Reproducibility: The setup must be straightforward to reproduce and redeploy, both for continued development within the project and for potential use by other researchers or partners in the collaboration.

3.2 Architecture

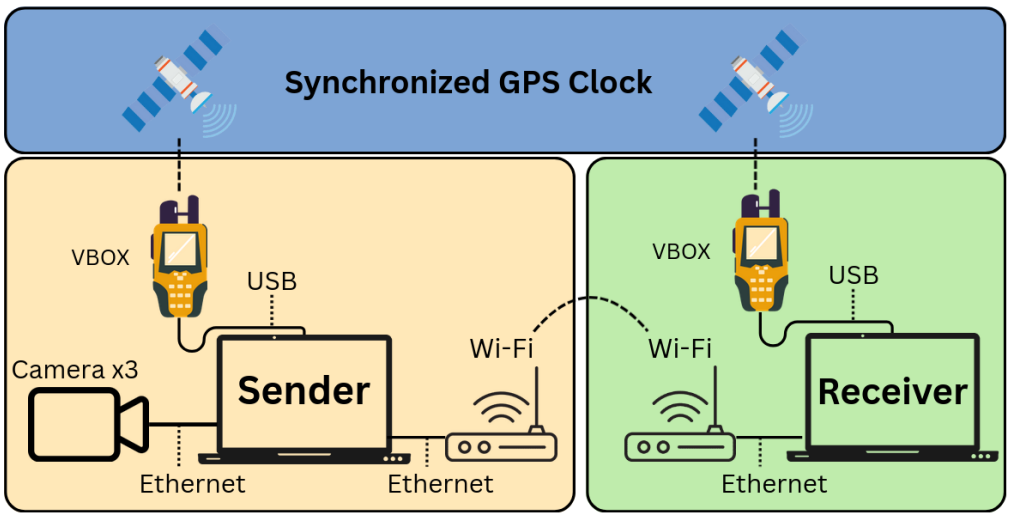

Figure 1: Physical Setup of Project

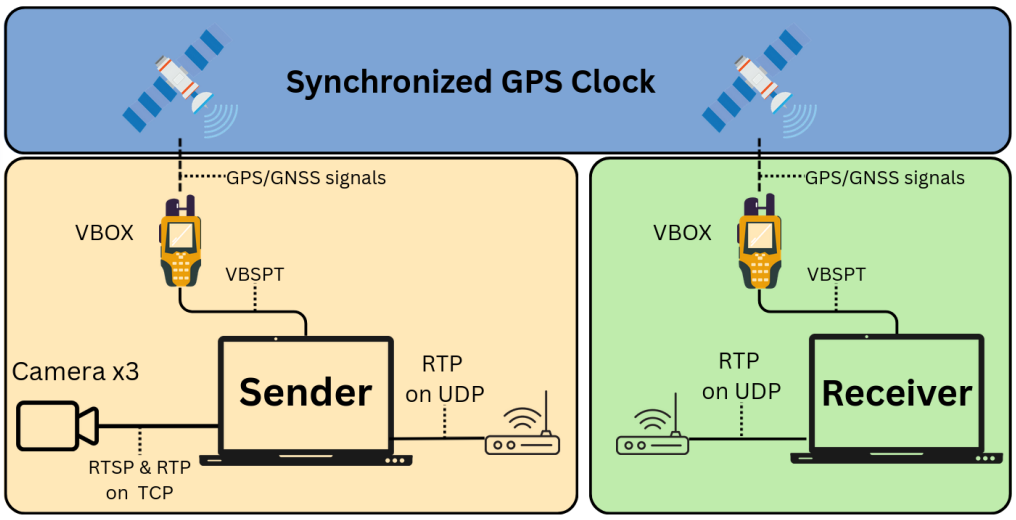

Figure 2: Application and Transport layer Setup of Project

Table 1: Setup for IP and Port Connection

Camera	Camera IP	Connection interface	Local IP	Receiver IP	RTP Port
Camera 1	192.168.0.100	eth0	192.168.0.50	10.238.111.249	5000
Camera 2	192.168.1.101	eth1	192.168.1.50	10.238.111.249	5002
Camera 3	192.168.2.102	enp0s31f6	192.168.2.50	10.238.111.249	5004

IP Cameras: Logged in with SADE, iVMS and Hikvision, to be set to standard IP address of 192.168.1.100, 101 and 102. This will remain IP address until manually overwritten.

Important to separate: Transport Layer: TCP, UDP Application Layer: RTSP, RTP TCP perfect, correct order and intact. UDP speedier but less reliable

Between Cam and Laptop, TCP. A NetCat TCP handshake to port 554 to check its availability.

Sender Computer Sets local IP address to 192.168.1.20 to make sure to be on the same subnet as the IP cameras. Pings local IP address of cameras connected to confirm possible connection. Checks RTSP port is available by using NetCat. NetCat sends a TCP handshake. RTSP message over TCP, asking for stream. Describe Stream, Camera describe, Setup up stream, Camera ready, Play stream. Setup because we ask for protocol TCP, we go Interleaved: RTSP and 2 Channels inside TCP connection. Channel 0 carries RTP (video data) Channel 1 carries RTCP (Control and monitoring QoS, Sync streams, provides feedback)

Real-Time Streaming Protocol control the stream. Using NetCat (needs download) to check if the RTSP port is open

Pulls the stream from the camera over RTSP/TCP, with no jitter buffer “latency = 0“ Camera sends: ———————————— (perfectly even) Network delivers: —— ———— ——-——— (uneven gaps)

Without buffer: —— ———— ——-——— (choppy, uneven) With buffer: waits. . . then ———————————— (smooth, even)

Because of local wired connection, the jitter is probably very low, and not adding jitter buffer will not affect much other than lowering latency.

rtph264depay, Strips the stream of RTP giving raw H.264 NAL units

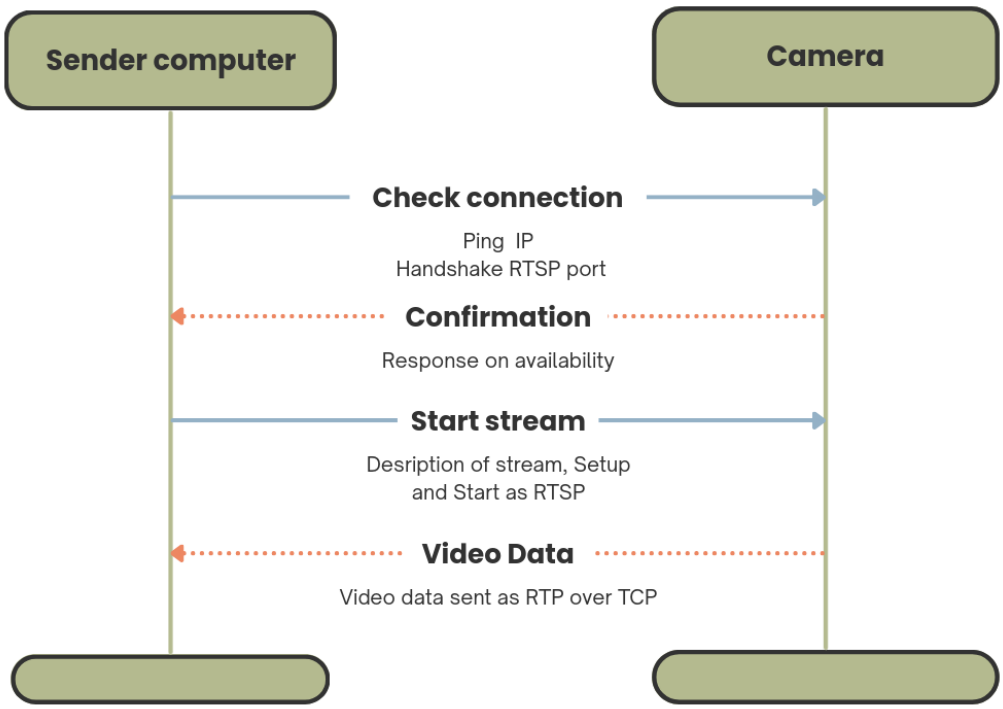

Figure 3: Sequence Diagram between Sender and Camera before active Stream

H.264 video is made up of NAL units (chunks of compromised video). RTP is rules how to split up big NAL units and how to assemble them again. Why Unpack and repack? Control over the stream — you get a clean interception point for your probe config-interval=1 — injecting SPS/PPS every frame so the receiver never gets lost Flexibility — you could insert transcoding, scaling, or other processing between depay and pay if needed later

What is a Probe? In GStreamer, a probe is essentially a spy you attach to a specific point in the pipeline. It lets you intercept data as it flows through, look at it, and optionally modify or drop it — all without interrupting the pipeline itself.

What is a Pad? Before understanding probes you need to understand pads — because probes attach to pads, not to elements directly. Every GStreamer element has pads — they are the connection points between elements:

rtph264pay, Package back into RTP packets, injects SPS/PPS every frame "config-interval-1" udpsink, sends packets to receiver with UDP

Receiver Computer: Listens to UDP port 5000 rtph264depay: Strips RTP again to get H.264 NAL units h264parse: Parses and structures the bitstream avdec-h264: Software decodes H.264 to raw frames (this is the expensive step) autovideosink: Displays on screen (sync=false = no clock sync, lowest latency)

3.3 Pipeline Design

3.4 Timestamps and Latency Measurement Design

3.5 Wi-Fi Network Setup

4 Methodology

4.1 Literature and Knowledge acquisition

The knowledge needed to carry out this thesis was acquired through an iterative process that combined systematic literature review with hands-on practical exploration. This approach, often referred to as an Iterative Literature and Practice-Driven Approach, is characterized by a continuous cycle where theory and practice inform each other rather than following a strict linear sequence [Wohlin et al. (2012)]. Instead of completing all literature review before beginning practical work, the two activities run in parallel, with new practical challenges prompting targeted literature searches and new theoretical insights shaping the direction of experiments. This makes it particularly well suited for applied engineering research where the problem space is not fully known at the outset and understanding develops gradually through doing [Hevner et al. (2004) Saunders et al. (2019)].

The process began by identifying the core knowledge areas required before any development could start. This included understanding the operational context of remote train operation and Automatic Train Operation, the fundamentals of video streaming and relevant transport protocols, and the capabilities and structure of GStreamer as a framework. These areas formed the initial scope of the literature review and gave enough grounding to begin practical work.

Academic literature was gathered primarily through Google Scholar and IEEE Xplore, using search terms related to low-latency video streaming, remote train operation, GStreamer pipeline design, and network performance for real-time applications. Sources were selected based on relevance to the problem, publication quality, and recency where applicable. In addition to academic papers, official technical documentation was used extensively, particularly the GStreamer documentation and protocol specifications for RTP and UDP, which provided the practical detail that academic sources alone could not cover.

Once a sufficient theoretical foundation was in place, practical exploration began through hands-on work with GStreamer. Building and testing pipelines from the command line and through Python quickly revealed challenges that existing literature did not fully address, such as specific configuration trade-offs, unexpected latency behavior, and hardware compatibility issues. Each of these practical obstacles prompted a return to the literature, this time with more precise and targeted questions. New findings were then brought back into the development process, tested, and evaluated against real behaviour.

This cycle of reading, building, observing, and searching again repeated throughout the entire development phase. The result is that the theoretical foundation of this thesis did not precede the practical work, but grew alongside it, shaped by the specific demands of the problem at hand.

4.1.1 Search Strategy and Databases

A multi-platform search approach was utilized to retrieve a wide array of high-quality sources. The primary databases included:

- Scopus: For retrieving peer-reviewed, impactful scientific articles.
- Google Scholar: For broader academic and institutional literature.
- GStreamer docs: For specifics on GStreamer development.

4.1.2 Inclusion Criteria

To ensure the study is based on the relevant and current information, specific criteria were applied to filter the search results. Given the rapid pace of technological change, a focus was placed on

recent publications. Where modern papers have referenced to older papers, I have include some as to say and show where certain numbers are coming from and how they have influenced results.

Table 2: Inclusion Criteria for Literature Review

ID	Inclusion Criteria (IC)
IC1	Publication Date: Between 2020-2026
IC2	Language: Written in English
IC3	Document Type: Primarily "Article", "Conference Paper" or "Thesis"

4.1.3 Search Queries and Refinement

Several digital tools as mentioned below were used for this paper. The main purpose of the usage was to enhance clarity, assisting in latex, and in general increase quality. Specifics include writing reference list in correct format. Structuring sentences and paragraphs. Checking for typos and grammatical errors.

- OpenAI's ChatGPT: Employed as a helpful resource for LaTeX formatting suggestions, generating structured content (tables and lists), translating between Norwegian and English, and reviewing text for synonyms and restructuring ideas to ensure arguments were effectively communicated.
- Google Gemini: Used for similar fields as ChatGPT, but also for general proofreading, and refining sentence structure and tone to maintain a high standard of academic writing.
- Anthropic's Claude.ai for discussion and development of script syntax.
- Visual Studio Code: was used as the main LaTeX editor because of its powerful extensions for LaTeX support, syntax highlighting, and version control integration.

4.2 Validity

4.3 Reliability

4.4 Research Design

This thesis follows an applied research approach, combining Design Science Research (DSR) and experimental research to address both the development and evaluation aspects of the work. This combination is well suited for engineering projects where the goal is to produce a functional artifact while also generating knowledge through systematic testing and measurement.

The DSR component governs the design and development of the video streaming pipeline. Following the principles outlined by Hevner et al. [source], the research contribution is embedded in the artifact itself, meaning the pipeline design, the architectural decisions made, and the reasoning behind them all form part of the research output. The problem is clearly grounded in practice, the solution is built and refined iteratively, and the artifact is evaluated against the requirements defined in the previous chapter.

The experimental research component covers the testing and evaluation phase. Drawing on the framework described by Wohlin et al. [source], the pipeline is tested across a range of configurations in a controlled setting. Variables such as codec choice, payload settings, and network conditions are systematically varied, and end-to-end latency is measured and compared across configurations. This allows conclusions to be drawn about which configurations perform best and what trade-offs exist between latency and video quality.

Together these two approaches reflect the dual nature of the thesis: it is both a development project and a measurement study, and the methodology is designed to support both goals.

4.5 Development

4.5.1 GStreamer Pipeline Development

4.6 Experimental Methodology

4.6.1 Latency Measurement

5 Results

6 Conclusion

6.1 Evaluation of latency